

Transformers

Attention Is All You Need — Study Guide

Phase 3 of your LLM learning path. This guide covers the five core concepts from the original 2017 paper, with formulas, architecture diagrams, and PyTorch project guidance.

Self-Attention	Multi-Head Attention	Positional Encoding	Encoder	Decoder
----------------	----------------------	---------------------	---------	---------

1. Architecture Overview

The Transformer processes an input sequence through an **Encoder** stack and generates output via a **Decoder** stack. Both stacks are composed of identical layers stacked N times (N=6 in the original paper).

ENCODER	DECODER
Input embedding tokens → d_model vectors	Output embedding prev tokens → vectors
Positional encoding add position signal	Positional encoding add position signal
× N layers	× N layers
Multi-head self-attention attend to all tokens	Masked multi-head attention attend only to past
Add & Norm residual + layer norm	Add & Norm residual + layer norm
Feed-forward network transform each position	Cross-attention attend to encoder output
Add & Norm residual + layer norm	Add & Norm + FFN + Add & Norm
→ contextual representations	Linear + Softmax → next token

GPT = decoder-only · BERT = encoder-only · T5 / original paper = encoder-decoder

2. Self-Attention

Self-attention lets every token in a sequence attend to every other token. For each token it produces three vectors — **Query (Q)**, **Key (K)**, and **Value (V)** — from learned projection matrices.

The formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) \cdot V$$

Step-by-step

- 1 Project to Q, K, V**
Multiply the token embedding by weight matrices W_Q , W_K , W_V to get three vectors per token.
- 2 Compute raw scores**
 $\text{dot}(Q_i, K_j)$ for every token pair. High score = token j is relevant to token i .
- 3 Scale by $\sqrt{d_k}$**
Prevents dot products from growing too large in high dimensions, stabilising gradients.
- 4 Apply softmax**
Convert scores to weights that sum to 1.0 — the attention distribution.
- 5 Weighted sum of V**
Output = sum of Value vectors, weighted by attention. The token now 'contains' context.

Intuition

Sentence	The cat sat on the mat
Query token	"sat"
High attention to	"cat" (0.35), "mat" (0.32)
Low attention to	"the" (0.05), "on" (0.10)

3. Multi-Head Attention

A single attention head can only focus on one relationship type. Multiple heads run **in parallel**, each in a lower-dimensional subspace, each learning to attend to different patterns.

Formula

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) · W_O  
head_i = Attention(Q · W_Qi, K · W_Ki, V · W_Vi)
```

Step-by-step

- 1 Project into h subspaces**
Each head gets its own smaller W_Q , W_K , W_V . Original paper: $h=8$, $d_k=64$.
- 2 Run attention independently**
Each head computes scaled dot-product attention in its 64-dim subspace.
- 3 Concatenate outputs**
All h outputs are joined back into a single vector of size $d_{\text{model}}=512$.
- 4 Final projection W_O**
A learned matrix maps the concatenation back to d_{model} dimensions.

What each head learns

Head type	What it attends to
Syntactic	Subject–verb agreement, grammatical dependencies
Coreference	Pronouns linked to their antecedents
Local adjacency	Nearby tokens — local phrase structure
Semantic similarity	Tokens with related meaning
Long-range	Dependencies across distant positions
Negation scope	Negative words and what they modify

4. Positional Encoding

Attention is **order-blind** — without positional information, "cat bit dog" and "dog bit cat" would produce identical embeddings. The solution: add a fixed positional signal to each token embedding.

The formula

$$PE(pos, 2i) = \sin(pos / 10000^{(2i / d_{model})})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i / d_{model})})$$

Why sine and cosine?

1 Unique per position

Each position gets a different vector — no two positions share the same encoding.

2 Bounded values

sin/cos always in $[-1, +1]$, so they don't distort the embedding magnitudes.

3 Relative position learnable

$PE(pos+k)$ can be expressed as a linear function of $PE(pos)$, so the model can learn relative offsets.

4 Different frequencies per dimension

Low dims oscillate fast (nearby positions differ sharply); high dims change slowly (long-range structure).

Visualising the pattern

pos \ dim	0	2	4	8	16	32	64	128
0	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
1	+0.84	+0.82	+0.80	+0.76	+0.68	+0.53	+0.31	+0.10
2	+0.91	+0.94	+0.96	+0.99	+1.00	+0.90	+0.59	+0.20
5	-0.96	-0.99	-1.00	-0.93	-0.57	+0.32	+1.00	+0.48
10	-0.54	-0.22	+0.12	+0.69	+0.94	-0.61	-0.02	+0.84
20	+0.91	+0.43	-0.24	-1.00	+0.65	-0.97	+0.04	+0.91

$\sin(pos/10000^{(d/512)})$ for even dimensions. Values in $[-1, +1]$.

5. Encoder & Decoder

Encoder — understanding input

The encoder processes the **entire** input sequence simultaneously (bidirectional). Every token can attend to every other token. The output is a set of rich contextual representations — one vector per input token — that the decoder will later consult.

- 1 Input embedding + positional encoding**
Convert tokens to d_{model} vectors and add position signals.
- 2 Multi-head self-attention**
Each token attends to all others — full bidirectional context.
- 3 Add & Norm**
Residual connection ($x + \text{sublayer}(x)$) + Layer Normalisation.
- 4 Feed-forward network**
Two-layer MLP applied to each position independently. $d_{\text{ff}} = 4 \times d_{\text{model}}$.
- 5 Repeat $\times N$**
Stack $N=6$ identical layers. Deeper layers capture increasingly abstract patterns.

Decoder — generating output

The decoder generates tokens **one at a time** (autoregressive). It has two additional mechanisms not found in the encoder.

- 1 Masked multi-head self-attention**
Can only attend to already-generated tokens (causal mask). Prevents peeking at future outputs during training.
- 2 Cross-attention (encoder–decoder attention)**
Queries from the decoder; Keys and Values from the encoder output. This is how the decoder reads the input.
- 3 Feed-forward network + Add & Norm**
Same as encoder. Each sub-layer followed by residual + LayerNorm.
- 4 Linear + Softmax**
Project $d_{\text{model}} \rightarrow$ vocabulary size, then softmax $\rightarrow$ probability distribution over next token.

Model variants

Model	Stack used	Training objective	Best for
GPT / GPT-4	Decoder only	Next-token prediction	Text generation, chat
BERT	Encoder only	Masked LM + NSP	Classification, NER, QA
T5 / BART	Encoder–Decoder	Seq2seq denoising	Translation, summarization
Original paper	Encoder–Decoder	Machine translation	WMT EN→DE/FR tasks

6. PyTorch Project — Build from Scratch

Build a small Transformer step-by-step. Each module maps directly to one of the concepts above.
Recommended dataset: Multi30k EN→DE translation (~30k sentence pairs).

Recommended build order

1	Token embedding layer nn.Embedding(vocab_size, d_model). Maps integer token IDs to d_model-dim vectors.
2	Positional encoding Fixed sin/cos matrix, added (not concatenated) to the embedding output.
3	Scaled dot-product attention Function: takes Q, K, V tensors, returns weighted V. Apply optional causal mask.
4	Multi-head attention module nn.Linear projections for Q/K/V per head, call attention, concat, project with W_O.
5	Encoder block MultiHeadAttention → Add & Norm → FFN (two nn.Linear + ReLU/GELU) → Add & Norm.
6	Decoder block Masked MHA → Add & Norm → Cross-attention → Add & Norm → FFN → Add & Norm.
7	Full encoder / decoder stacks nn.ModuleList of N encoder blocks; N decoder blocks consuming encoder output.
8	Training loop Cross-entropy loss on next-token prediction, Adam optimizer (lr=0.0001, betas=(0.9,0.98)).

Key hyperparameters (original paper)

Parameter	Value	Notes
d_model	512	Embedding / hidden dimension
h (heads)	8	Each head: d_k = d_v = 64
N (layers)	6	Encoder and decoder each
d_ff	2048	Inner FFN dimension (4× d_model)
Dropout	0.1	Applied after each sub-layer
Max seq len	512	For positional encoding
Optimizer	Adam	Warmup + inverse sqrt schedule

PyTorch skeleton

```
import torch, torch.nn as nn, math  
  
  
class MultiHeadAttention(nn.Module):
```

```
def __init__(self, d_model=512, h=8):
    super().__init__()
    self.h, self.d_k = h, d_model // h
    self.W_q = nn.Linear(d_model, d_model)
    self.W_k = nn.Linear(d_model, d_model)
    self.W_v = nn.Linear(d_model, d_model)
    self.W_o = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, mask=None):
        B, T, _ = q.shape
        # project & split into heads
        Q = self.W_q(q).view(B,T,self.h,self.d_k).transpose(1,2)
        K = self.W_k(k).view(B,T,self.h,self.d_k).transpose(1,2)
        V = self.W_v(v).view(B,T,self.h,self.d_k).transpose(1,2)
        # scaled dot-product attention
        scores = Q @ K.transpose(-2,-1) / math.sqrt(self.d_k)
        if mask is not None: scores = scores.masked_fill(mask==0, -1e9)
        attn = scores.softmax(-1)
        out = (attn @ V).transpose(1,2).reshape(B,T,-1)
        return self.W_o(out)
```

Reference implementations

Resource	URL	Notes
Andrej Karpathy — nanoGPT	github.com/karpathy/nanoGPT	Minimal GPT, ~300 lines
Harvard NLP — The Annotated Transformer	nlp.seas.harvard.edu/annotated-transformer	Line-by-line paper walkthrough
PyTorch docs	pytorch.org/docs/stable/nn.html#transformer-layers	Built-in nn.Transformer, nn.MultiheadAttention

Quick-Reference Cheat Sheet

Term	Formula / Value	Plain-English meaning
Self-attention	$\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$	Each token looks at all others, weighted by relevance
d_k scaling	divide by $\sqrt{d_k}$	Prevents dot products from exploding in high dims
Multi-head	$\text{Concat}(\text{head}_1..h) \cdot W_0$	h parallel attention ops, each in a subspace
Positional enc.	sin/cos at $2i/2i+1$ dims	Fixed wave pattern encodes token order
Add & Norm	$\text{LayerNorm}(x + \text{sublayer}(x))$	Residual skip + normalize — stabilises deep nets
FFN	$\max(0, xW_1+b_1)W_2+b_2$	Two linear layers with ReLU, applied per position
Cross-attention	Q from decoder, K/V from encoder	Decoder reads the encoded input
Causal mask	mask future positions to $-\infty$	Decoder can't peek at tokens not yet generated
Softmax	$\exp(x_i) / \sum(\exp(x))$	Converts logits to a probability distribution